

EXAMPLES

DATA DB 5 DUP(12) ; 5 Bytes containing hex 0c0c0c0c0c
 DATA DB 10 DUP(?) ; 10 Words Uninitialized
 DATAZ DB 3 DUP(5 DUP(4)) ; 44444 44444 44444

CHARACTER STRINGS

Character Strings are used for descriptive data. Consequently DB is the conventional format for defining character data of any length

An Example is

DB 'Computer City' DB "Hello World"
 DB "St. Xavier's College"

2. NUMERIC CONSTANTS

#BINARY : VAL1 DB 10101010B
 #DECIMAL : VAL1 DB 230
 #HEXADECIMAL : VAL1 DB 23H

8086 Microprocessor Instruction Sets**1. Data Transfer Instructions****1.1 General Purpose Byte or Word Transfer Instructions**

Instructions	Comments
MOV MOV Destination, Source MOV CX, 04H	Copy byte or word from specified source to specified destination.
PUSH PUSH Source PUSH BX	Copy specified word to top of stack.
POP POP Destination POP AX	Copy word from top of stack to specified location.
XCHG XCHG Destination, Source XCHG AX, BX	Exchange word or byte.
XLAT	Translate a byte in AL using a table in memory. It first adds AL + BX to form memory address. It then copies the content into AL.

1.2 Simple Input and Output Port Transfer Instructions

INSTRUCTIONS	COMMENTS
IN IN AX, Port_Addr IN AX, 34H	Copy a byte or word from specified port to accumulator.
OUT OUT Port_Addr, AX OUT 2C H, AX	Copy a byte or word from accumulator to specified port.

1.3 Special Address Transfer Instructions

INSTRUCTIONS	COMMENTS
LEA LEA Register, Source LEA BX, PRICE	Load effective address of operand into specified register.
LDS LDS Register, Source LDS BX, [4326H]	Load DS register and other specified register from memory.
LES	Load ES register and other specified register from memory.

1.4 Flag transfer instructions

INSTRUCTIONS	COMMENTS
LAHF	Copy to AH with the low byte of the flag register.
SAHF	Stores AH register to low byte of flag register.
PUSHF	Copy flag register to top of stack.
POPF	Copy word at top of stack to flag register.

2. Arithmetic instructions

2.1 Addition Instructions

INSTRUCTIONS	COMMENTS
ADD ADD Destination, Source ADD AL, 74H	Add specified byte to byte or word to word.
ADC ADC Destination, Source ADC CL, BL	Add byte + byte + carry flag Add word + word + carry flag
INC INC Register INC CX	Increment specified byte or word by 1.
AAA	ASCII adjust after addition.
DAA	Decimal adjust after addition.

2.2 Subtraction Instructions

INSTRUCTIONS	COMMENTS
SUB SUB Destination, Source SUB CX, BX	Subtract byte from byte or word from word.
SBB SBB Destination, Source SBB CH, AL	Subtract byte and carry flag from byte. Subtract word and carry flag from word.
DEC DEC Register DEC CX	Decrement specified byte or word by 1.
NEG NEG Register NEG AL	Form 2's complement.

CMP CMP Destination, Source CMP CX, BX CF ZF SF 0 1 0 CX=BX 0 0 0 CX>BX 1 0 1 CX<BX	Compare two specified bytes or words.
AAS	ASCII adjust after subtraction.
DAS	Decimal adjust after subtraction.

2.3 Multiplication Instructions

INSTRUCTIONS	COMMENTS
MUL MUL Source MUL CX	Multiply unsigned byte by byte or unsigned word by word. When a byte is multiplied by the content of AL, the result is kept into AX. When a word is multiplied by the content of AX, MS Byte in DX and LS Byte in AX.
IMUL IMUL Source IMUL CX	Multiply signed byte by byte or signed word by word.
AAM	ASCII adjust after multiplication. It converts packed BCD to unpacked BCD.

2.4 Division Instructions

INSTRUCTIONS	COMMENTS
DIV DIV Source DIV BL DIV CX	Divide unsigned word by byte Divide unsigned word double word by byte. When a word is divided by byte, the word must be in AX register and the divisor can be in a register or a memory location. After division AL(quotient) AH(remainder) When a double word is divided by word, the double word must be in DX : AX pair and the divisor can be in a register or a memory location. After division AX(quotient) DX (remainder)
AAD	ASCII adjust before division
CBW	BCD to binary convert before division.
CWD	Fill upper byte of word with copies of sign bit of lower byte. Fill upper word of double word with sign bit of lower word.

3. Bit Manipulation Instructions

3.1 Logical Instructions

INSTRUCTIONS	COMMENTS
NOT NOT Destination NOT BX	Invert each bit of a byte or word.
AND AND Destination, Source AND BH, CL	AND each bit in a byte/word with the corresponding bit in another byte or word.
OR OR Destination, Source OR AH, CL	OR each bit in a byte or word with the corresponding bit in another byte or word.
XOR XOR Destination, Source XOR CL, BH	XOR each bit in a byte or word with the corresponding bit in another byte or word.
TEST TEST Destination, Source TEST AL, BH	AND operands to update flags, but don't change the operands.

3.2 Shift Instructions

INSTRUCTIONS	COMMENTS
SHL/SAL SAL Destination, Count SHL Destination, Count CF ← MSB ← LSB ← 0	Shift Bits of Word or Byte Left, Put Zero(s) in LSB.
SHR SHR Destination, Count 0 → MSB → LSB → CF	Shift Bits of Word or Byte Right, Put Zero(s) in MSB.
SAR SAR Destination, Count MSB → MSB → LSB → CF	Shift Bits of Word or Byte Right, Copy Old MSB in to New MSB.

3.3 Rotate Instructions

INSTRUCTIONS	COMMENTS
ROL	Rotate Bits of Byte or Word Left, MSB to LS and to CF.
ROR	Rotate Bits of Byte or Word Right, LSB to MSB and to CF.
RCL	Rotate Bits of Byte or Word Left, MSB to CF and CF to LSB.
RCR	Rotate Bits of Byte or Word Right, LSB to CF and CF to MSB.

4. Program Execution Transfer Instructions

4.1 Unconditional Transfer Instruction

INSTRUCTIONS	COMMENTS
CALL	Call a Sub program/Procedure.
RET	Return From Procedure to Calling Program.
JMP	Go to Specified Address to Get Next Instruction (Unconditional Jump to Specified Destination).

4.2 Conditional Transfer Instruction

INSTRUCTIONS	COMMENTS
JA/JNBE	Jump if Above/Jump if Not Below or Equal.
JAЕ/JNB	Jump if Above or Equal/Jump if Not Below.
JB/JNAE	Jump if Below/Jump if Not Above or Equal.
JBE/JNA	Jump if Below or Equal/Jump if Not Above.
JC	Jump if Carry Flag CF = 1.
JE/JZ	Jump if Equal/Jump if Zero Flag (ZF=1).
JG/JNLE	Jump if Greater/Jump if Not Less than or Equal.
JGE/JNL	Jump if Greater than or Equal/Jump if Not Less than.
JL/JNGE	Jump if Less than/Jump if Not Greater than or Equal.
JLE/JNG	Jump if Less than or Equal/Jump if Not Greater than.
JNC	Jump if No Carry i. e. CF = 0
JNE/JNZ	Jump if Not Equal/Jump if Not Zero (ZF=0)
JNO	Jump if No Overflow.
JNP/JPO	Jump if Not Parity/Jump if Parity Odd.
JNS	Jump if Not Sign (SF=0)
JP/JPE	Jump if Parity/Jump if Parity Even(PF=1)
JS	Jump if Sign (SF=1)

4.3 Iteration Control Instructions

INSTRUCTIONS	COMMENTS
LOOP	Loop Through a Sequence of Instructions Until CX = 0.
LOOPE/LOOPZ	Loop Through a Sequence of Instructions While ZF = 1 and CX! = 0.
LOOPNE/LOOPNZ	Loop Through a Sequence of Instruction While ZF = 0 & CX! = 0.
JCXZ	Jump to Specified Address if CX = 0.

4.4 Interrupt Instructions

INSTRUCTIONS	COMMENTS
INT	
INT0	
IRET	Interrupt Program Execution if OF=1 Return From Interrupt Service Procedure to Main Program.

5. Process or Control Instructions

5.1 Flag Set/Clear Instruction

INSTRUCTIONS	COMMENTS
STC	Set Carry Flag CF to 1.
CLC	Clear Carry Flag to 0.
CMC	Complement the State of CF.
STD	Set Direction Flag to 1.
CLD	Clear Direction Flag to 0.
STI	Set Interrupt Flag to 1. (Enable INTR Input).
CLI	Clear Interrupt Enable to 0

5.2 No Operation Instruction

INSTRUCTIONS	COMMENTS
NOP	No Action Except Fetch and Decode.

5.3 External Hardware Synchronization INST

INSTRUCTIONS	COMMENTS
HLT	Halt (Do Nothing) Until Interrupt or Reset.
WAIT	Wait Until Signal On the TESTP in is Low.
ESC	Escape to External Coprocessor Such as 8087 or 8089.
LOCK	Prevents Another Processor From Taking the Bus While the Adjacent Instruction Executes.

6. String Instructions

INSTRUCTIONS	COMMENTS
REP	Repeat Instruction Until CX = 0.
REPE/REPZ	Repeat if Equal/Repeat if Zero
REPNE/REPNZ	Repeat if Not Equal/Repeat if Not Zero.
MOVS/MOVS/MOVSW	Move Byte or Word From One String to Another.
COMPS/COMPSB/COMPSW	Compare Two String Bytes or Two String Words.
SCAS/SCASB/SCASW	Compares a Byte in AL or Word in AX With a Byte or Word Pointed By DI in ES.

DOS Function Codes

DOS INT 21h

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		

3. **Sum of series of 10 numbers and store result in memory location total.**

Title Sum of series

.model small

.org 100h

.data

List db 12,34,56,78,98,01,13,78,18,36

Total dw?

.code Mainproc

MOV AX, @data

MOV DS, AX

~~AX~~ MOV AX, 0000H

MOV CX, 0AH ; counter

MOV BL, 00H; to count carry

MOV SI, offset List

Back: ADD AL,[SI]

JC Label

Back1: INC SI

LOOP Back

MOV Total, AX

MOV Total+2, BL

MOV AH, 4CH

INT 21H

Label: INC BL

JMP Back1 Mainendp EndMain

4. **Write a program to find Largest No. in a block of data. Length of block is 0A. Store the maximum in location result.**

.model small

.data

List db 01h,12h,03h,04h,01h,02h

Result db ?

.code

main sumproc

MOV AX, @data ; Initialize Data segment

MOV DS, AX

LEA SI, List; Load SI with the offset address of list

108 Microprocessor

```

MOV AX,0000h; Clear accumulator
MOV CX,06h; Load CX with count
MOV DX,0000h; Clear DX
CLC; clear carry
up:  ADD AX,[SI]; Add the first element with accumulator
      INC SI; Increment SI twice
      DEC CX; Decrement the count
      JNZ up; If CX≠0, repeat the loop
      JNC down; else if carry =0, jump to label down
      INC DX; if carry=1, increment dx
down: MOV Result, AL ;Store the result in memory pointed to by SI
      MOV Sum+1,DL; Store carry in next memory location
      MOV AH, 4CH; Terminate the program
      INT 21h
      ENDP
      ret

```

5. Find number of times letter 'e' exist in the string 'exercise', Store the at memory count.

Title string operation

```

.model small
.stack 100h
.data
String db 'exercise', $
Count db ?
Length db $.String
.code Mainproc
MOV AX, @data
MOV DS, AX
MOV AL,00H
MOV SI, offset String
MOV CX, Length
Back: MOV BH, [SI]
      CMP BH, 'e'
      JNZ Label
      IN CAL
      INC AL

```


Label: INC SI

LOOP Back

MOV count, AL

MOV AH, 4CH

INT 21H

Main endp

EndMain

8086 Program

12. Program to read and display a character from the keyboard

.model small

.stack

.data

.code

MOV AX, @Data

;Initialize

MOV DS, ax

; the data segment

MOV AH, 01h

; DOS function to get a

INT 21H

MOV DL, AL

MOV AH, 02h

; DOS function to get a
; character from keyboard

INT 21H

MOV AH, 4CH

;Terminate

INT 21H

; the program

end

13. **Program to read a character from the keyboard with buffered input**

.model small

.stack

.data

buff db 30 ;max length passed to the function
db 0 ;actual length returned by function
db 30 dup(0) ;space for storing returned string from function

.code

MOV AX, @Data ;Initialize
MOV DS, AX ; the data segment
MOV AH, 0AH ; DOS function to
LEA DX, BUFF ; read string from the keyboard
INT 21H
MOV AH, 4CH ; Terminate
INT 21H ; the program

END

14. **Write a program to compare two string data.**

STRING2 DB 'EMPPY\$'

LENGTH DW STRING2-STRING1

NEQU DB 'STRINGS ARE UNEQUAL\$'

EQUA DB 'STRINGS ARE EQUAL\$'

.CODE

MOV AX,@DATA ; Initialize DS through AX register

MOV DS,AX

MOV SI, OFFSET STRING1 ; Initialize SI by the offset of STRING1

MOV DI, OFFSET STRING2 ; Initialize DI by the offset of STRING2

MOV CX,LENGTH

MOV BX,00H

BACK: MOV AL,[SI]

CMP AL,[DI]

JNZ SKIP

INC BX

SKIP: INC SI

INC DI

LOOP BACK

CMP BX,LENGTH

JZ EQUAL

116 Microprocessor

MOV DX,OFFSET NEQU

MOV AH,09H

INT 21H

JMP LAST

EQUAL: MOV DX,OFFSET EQUA

MOV AH,09H

INT 21H

LAST:MOV AH,4CH

INT 21H

END

8086 MP :-

* Assembling & linking :-

Machine language :- There is only one programming language that any computer can actually understand and execute, it's on native binary machine code. This is the lowest possible language to write a computer program i.e. binary code. But binary code is not native to humans and very easy for error to occur in the program. The bugs (small mistakes) are not easy to determine in a pool of binary digits. It is impossible to memorize thousands of binary instruction code.

Assembly language :-

- In assembly language programs are represented by certain words representing the operation of instruction.
- Program gets easier.
- The words (2-4 letter) is used to represent each instruction are called mnemonics.
- Assembly language statements are generally ^{written} ~~return~~ in a standard form that has four fields. i.e. 1. label field
- 2) Mnemonics or op-code field.

(3) Operation field (opcode)

(4) Comment field.

Label	Mnemonic	Operand	Comment
start:	MVI	A, 10H	: move 10H into accumulator
	MVI	B, 20H	: move 20H into register B
	ADD	B	: add the contents of register B with accumulator

These codes are converted into machine code by assembler i.e. machine language program (object code). Assembly language programs are called source code.

The process of converting source code into object code is called compilation and assembler does it. The process of converting object into executable format is called linking and linker does it.

Source code $\xrightarrow[\text{assembler does it}]{\text{Compilation}}$ object code (machine code) $\xrightarrow[\text{linker does it}]{\text{linking}}$ Executable format

Assembler directives:-

They are number of statements that enable or help to control the way the source program is assembled and ~~list~~ lists.

int	2 byte
Float	4 byte
double	8 byte
Char	1 byte

(1) The page & title listing directives :-

They control the format of a listing of an assembled program. The page directives defines the maximum number of lines to list as a page and the maximum number of character as a line.

eg:- PAGE [length] [width]
Default page [50] [80]

Title gives title, and place the title on second line of each page of the program.

(2) SEGMENT & END's directives :- It is used to indicate the start of a logical statement. It is the name given to the segment.

It gives the start of a segment or a STACK, data and code. eg:-

ARRAY 1 SEGMENT

MOV AX, BX

ADD AX, BX

ARRAY 1 END's

} logical segment

(3) ASSUME directive :- Here segment name is ARRAY 1 the assembler will assign a numeric value to ARRAY 1 corresponding to the base value of the data segment. The data array or an instruction sequence between the segment and END's directive is called a logical segment.

(3) ASSUME directive:-

It is used to tell the assembler the purpose of each segment in the program. eg:-

ASSUME ss: stack name, DS: data segment,
CS: code segment.

(4) BYTE, DATA WORD, DOUBLE WORD DIRECTIVES

Here, data byte (DB) ^{8bit}, DATA WORD (DW) ^{16bit}, DOUBLE WORD DIRECTIVES (DD) ^{32bit} are of size 8-bit, 16 bit, 32 bit respectively.

Eg:- COUNT DB 0 : Count is defined as data byte

COMP DW 25AAH : COMP is variable declared as a word with content 25AAH.

FORB DD 08H : double word.

Here DB, DW, & DD are the directives used to assign names and specific data types for variable in a program. After execution of comp DW 25AAH, the assembler assign AAH to the offset name COMP and 25 as to offset name COMP + 1.

(5) EQU directive:-

It is the short form of equivalent, used to assign a name to constant eg:-

PROD EQU 55H ; Replace PROD by 55H
MOV BX, PROD ; same as MOV BX, 55H.

⑥ DUP directive

It is the short form of duplicate. The DUP directives can be used to initialize several location to zero.

PRICE DW 4 DUP (0);

This instruction reserves 4 words starting at the offset price in data segment and initializes them to zero.

If we need not to be initialized then (?) this is used.

WEIGHT DB 100 (?) ^{bytes}

This instruction reserves 100 ~~bits~~ of uninitialized data space to an offset WEIGHT in DS.

⑦ INT 21H function
- 01H, 02H, 09H, 0AH, 4CH
4CH

⑧ Write a program in 8086 to display a string.
TITLE to display a string

- DOSSEG
- MODEL SMALL
- STACK 64H
- DATA

Addressing Modes in 8086.

(i) Register Addressing Mode.

In this mode, the operand is specified in register i.e. operands are present in specific register.

Example:

MOV AH, BL

MOV CX, DX

Here,

The operand is present in register BL which is transferred to register AH.

(ii) Immediate Addressing Mode.

In this addressing mode, the operand is present in the instruction.

Example:

MOV AH, 45 H

MOV BX, 2070 H

(iii) Direct Addressing Mode.

In this addressing mode, the location of operand is specified in a instruction.

Example:

→ ADD AX, [5000 H]

Here,

data resides in memory location in the data segment whose effective address may be computed using 5000 H as the offset address and content of DS as segmented address.

$$\text{Effective Address (EA)} = \text{DS} + 5000.$$

$$= \text{DS} \times 10 \text{ H} + 5000$$

$$= 20000 + 5000 = 25000 \text{ H} //$$

→ MOV AL, [1234 H]

Effective Address = AS + 1234.

(iv) Indirect Memory Addressing Mode.

In this mode, effective address of operand is taken directly from one of the base register (BX or BP) or index register (SI or DI).

Example:

→ MOV CX, [BX]

If AS = 1000H, [BX] = 0008

Effective Address = 10000 + 0008 = AS + [BX]

= 10008 H //

→ ADD, CX, [SI]

E.A. = AS + [SI]

= 10000 + [SI]

(v) Based Addressing Mode.

In this mode, the effective address is the sum of the displacement and the contents of the register BX or BP.

Example:

MOV AX, ^{displacement value} 8 _{16 bits} [BX]

Effective Address = AS + [BX] + 0008

(vi) Indexed Addressing Mode:

In this addressing mode, the P.A. is calculated with the help of displacement value specified in a instruction and index register.

Example:

MOV AX, value [SI]

Physical Address = AS + value + [SI]

throughput

- does not decrease time

Page No.

Date: / /

(vii) Based Indexed Addressing Mode.

In this addressing mode, physical address is calculated using indexed register, based register and displacement value.

Example:

MOV AX, 4[BX][SI]

Physical Address = $AS + 0004 + [BX] + [SI]$

DOS interrupt signal function

INT 21H

It is a DOS interrupt signal. There are different functions of ~~It~~ INT 21H.

(1) MOV AH, 01

INT 21H

These two instructions together will accept one single character from keyboard and store it in AL register.

(2) MOV AH, 02H
INT 21H

These two instructions will display a single character stored in the AL register.

Example:

MOV AL, 41H : 41H is ascii value of 'A' so it display 'A'

MOV AH, 02H

INT 21H

(3) MOV AH, 09H
INT 21H

These two instructions will display an entire string stored in memory location. For this, these two conditions are a must.

- The offset address of the string has to be stored in AX register.

- The string must be terminated by a \$.

Example:

(4) MOV AH, 4CH or MOV AX, 4C00H
INT 21H

These two instructions will terminate the program execution.

(5) MOV AH, 0AH
INT 21H

To input a string, we use these two instructions.

WAP in 8086 to display a string "B.Sc.CSIT".

TITLE TO DISPLAY A STRING

BOSSEG

• MODEL SMALL

• STACK 64H

• DATA

2B = data byte
8 bit

Str1 DB 'BScCSIT', \$

• CODE

Main Program

MAIN PROC

Data segment
Initialization

MOV AX, @ DATA

MOV DS, AX

MOV AH, 09H

↗ Load Effective Address.

MOV AX, offset Str1 / LEA AX, Str1

INT 21H

MOV AH, 4CH

MOV AX, 4C00H

INT 21H

MAIN ENDP

END MAIN

Write an assembly program in 8086 to input a character and display it.

TITLE TO INPUT A CHARACTER AND DISPLAY

BOSSEG

• MODEL SMALL

• STACK 64H

• DATA

V21 DB

• CODE

MAIN PROC

MOV AX, @ DATA

MOV DS, AX

MOV AH, 01H

MOV AL, 'P'

/ MOV var, 'P'

MOV AL, var.

INT 21H

MOV AH, 02H

MOV DL, AL

INT 21H

MOV AH, 4CH

INT 21H

MAIN ENBP

END MAIN

Write an assembly language program to in 8086 to input a string and display it.

TITLE TO INPUT A STRING AND DISPLAY

OSSEG

• MODEL SMALL

• STACK 64H

• DATA

Data word
bit

Str ~~DB~~ DB

• CODE

MAIN PROC

MOV AX, @ DATA

MOV DS, AX

MOV AH, 0AH

~~INT 21H~~ MOV Str, DW 'NCCS College', \$

INT 21H

MOV AH, 09H

MOV AX, offset Str

INT 21H

MOV AH, 4CH

```
INT 21H
MAIN ENDP
END MAIN.
```

Q. Write an assembly language program to display string in reverse order.

TITLE TO DISPLAY STRING IN REVERSE ORDER.

BOSSEG

.MODEL SMALL

.STACK 64H

.DATA

String1 ^{DB} 'Assembly language program'; \$
Length ^{DB} \$ - String1

.CODE

~~MOV SI, offset String1~~

MAIN PROC

MOV AX, @DATA

MOV DS, AX

MOV AH, 09H

MOV SI, offset String1 : SI = 100

MOV CX, Length : CX = 08

ADD SI, CX : SI = SI + CX

= 100 + 08 = 108

PA = SI + DS
= 2000 + 108
= 2108

BACK: MOV AL, [SI]

MOV AH, 02H

INT 21H

DEC SI / DEC BX

LOOP BACK / JNZ BACK

MOV AH, 4CH

INT 21H

MAIN ENDP

END MAIN.

\$	08
V	09
L	0A
D	0B
M	0C
E	0D
S	0E
A	0F
100	2100

MOV DI, [BX, DI and 20 times 0.1

Pipelining

Pipelining is an implementation technique where multiple instructions are overlapped in execution.

The computer pipeline is divided in **stages**. Each stage completes a part of an instruction in parallel.

The stages are connected one to the next to form a pipe - instructions enter at one end, progress through the stages, and exit at the other end.

Pipelining does not decrease the time for individual instruction execution. Instead, it increases instruction throughput. The **throughput** of the instruction pipeline is determined by how often an instruction exits the pipeline.

Because the pipe stages are hooked together, all the stages must be ready to proceed at the same time. We call the time required to move an instruction one step further in the pipeline a *machine cycle*. The *length* of the machine cycle is determined by the time required for the slowest pipe stage.

The pipeline designer's goal is to balance the length of each pipeline stage. If the stages are perfectly balanced, then the time per instruction on the pipelined machine is equal to

Time per instruction on nonpipelined machine / Number of pipe stages

Under these conditions, the speedup from pipelining equals the number of pipe stages. Usually, however, the stages will not be perfectly balanced; besides, the pipelining itself involves some overhead.

Instruction Pipeline

Any architecture can be pipelined by making each clock cycle into a pipe stage.

Clock#	1	2	3	4	5	6	7
Instruction i	Fetch	Decode	Execute				
Instr. $i + 1$		Fetch	Decode	Execute			
Instr. $i + 2$			Fetch	Decode	Execute		
Instr. $i + 3$				Fetch	Decode	Execute	
Instr. $i + 4$					Fetch	Decode	Execute

Table 2.1: Instruction Pipeline

Here instruction i is fetched in clock #1. After it has been fetched it is decoded in clock #2 and at the same time next instruction i . e. instr. $i+1$ is fetched. At Clock#3, instruction i is executed, instr. $i+1$ is decoded and instr. $i+2$ is fetched and so on.

Hence at the end of clock #3, instruction i is executed. Sometime later at the end of clock #4 instruction $i+1$ is executed.

If we assume that unpipelined architecture took 3 ns then this pipelined architecture will take 1 ns to finish a stage (fetch, decode and execute).

Advantages of pipelining:

- ⌘ The execution unit always reads the next instruction byte from the queue in BIU. This is faster than sending out an address to the memory and waiting for the next instruction byte to come.
- ⌘ In short pipelining eliminates the waiting time of EU and speeds up the processing. The 8086 BIU will not initiate a fetch unless and until there are two empty bytes in queue. 8086 BIU normally obtains two instruction bytes per fetch.